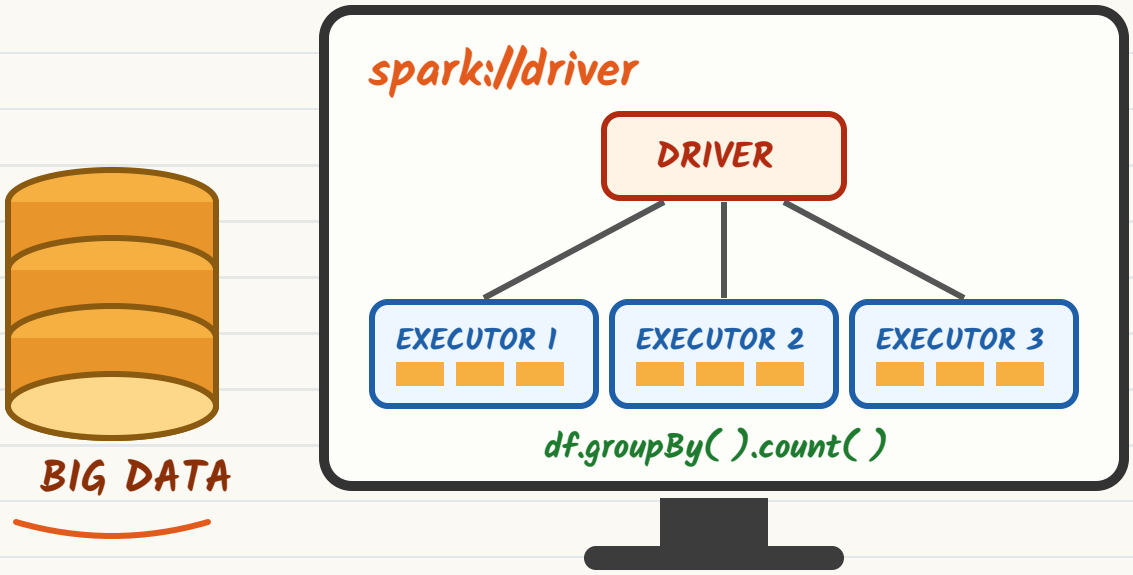
PySpark

INTERVIEW Q & A

≋ (120+ Questions with Answers) ≋



BEGINNER TO ADVANCED

SWIPE TO NEXT →

Spark & PySpark — The Warm-Up

Round 1 · Screening Questions

1 What is Apache Spark?

An open-source ***distributed processing engine*** for large-scale data. It splits work across a cluster, keeps intermediate results in memory, and exposes one engine for batch, SQL, streaming and ML.

2 Then what is PySpark?

The ***Python API*** for Spark. Your Python code talks to the JVM through Py4J. DataFrame code is translated to the same Catalyst plan as Scala — so no speed penalty. Only Python UDFs and RDD lambdas pay the serialization cost.

3 Why is Spark faster than Hadoop MapReduce?

MR writes to disk after *every* map and reduce. Spark builds a DAG of the whole job, keeps data in memory between stages, and only spills when it must — 10–100× on iterative work.

Point	Hadoop MapReduce	Apache Spark
Processing	Disk-based, map → reduce only	In-memory, full DAG of stages
Latency	High (minutes)	Low (sub-second to seconds)
APIs	Java, verbose	Python, Scala, Java, R, SQL
Workloads	Batch only	Batch + SQL + Streaming + MLlib + GraphX
Fault recovery	Re-run tasks from HDFS	Lineage — recompute lost partitions

Core Features 🚀

- ✓ Lazy evaluation + DAG optimisation
- ✓ In-memory caching & persistence
- ✓ Fault tolerance via lineage
- ✓ Polyglot APIs, one engine
- ✓ Runs on YARN, K8s, Standalone, Databricks

Where It's Used 🏭

- 🛒 ETL / ELT pipelines on TBs of data
- 📊 Building warehouse & lakehouse tables
- ⚡ Near real-time streaming pipelines
- 🤖 Feature engineering for ML

💡 Tip

Say "***unified engine***" in your first answer. Interviewers wait for it — batch, SQL, streaming and ML on one runtime is Spark's whole pitch.

⭐ Interview Tip

"Spark is in-memory so it never touches disk" is wrong. Shuffle writes always hit disk. Say "minimises disk I/O" instead.

Spark Architecture

Q4 · Explain Spark architecture

Driver — runs your `main()`, builds the DAG, splits it into stages/tasks and schedules them. Holds the SparkSession.

Cluster Manager — YARN / Kubernetes / Standalone. Hands out containers.

Executors — JVMs on worker nodes. They run tasks and hold cached data. One executor = many task slots (= cores).

Task — the smallest unit; 1 task processes 1 partition.

HOW A JOB FLOWS

Your PySpark code

↓

Unresolved Logical Plan

↓ Catalyst

Optimised Logical Plan

↓

Physical Plans → cost model

↓ Tungsten

RDDs → DAG → Stages

↓

Tasks run on Executors

Q5 · Job vs Stage vs Task

Unit	Created by
Job	One action (count, write, collect)
Stage	Split at every shuffle boundary
Task	One per partition inside a stage

Formula: 3 actions with 1 shuffle each → 3 jobs, 6 stages, and tasks = total partitions per stage.

Q6 · Client vs Cluster deploy mode

Aspect	Client	Cluster
Driver runs	On the submitting machine	Inside the cluster (AM container)
Use for	Notebooks, dev, debugging	Production scheduled jobs
If gateway dies	Job dies with it	Job survives

Q7 · What is Catalyst?

Spark's **query optimiser**. Does predicate pushdown, column pruning, constant folding and join reordering before any data moves. Tungsten then handles off-heap memory + whole-stage codegen.

⭐ Interview Tip

Draw it. Say the sentence **"one task per partition, one stage per shuffle, one job per action"** — that single line answers half the architecture round.

RDD vs DataFrame vs Dataset

Q8 · The classic comparison

Feature	RDD	DataFrame	Dataset
Abstraction	Distributed collection of objects	Distributed table with named columns	Typed DataFrame
Schema	None	Yes, known at runtime	Yes, compile-time typed
Optimiser	No Catalyst	Catalyst + Tungsten	Catalyst + Tungsten
In PySpark?	Yes	Yes — use this	Not available (Scala/Java only)
Best for	Unstructured data, low-level control	99% of real work	Type safety in Scala

Why no Dataset in Python? Python is dynamically typed, so compile-time type safety cannot be enforced. In PySpark a DataFrame is a `Dataset[Row]` under the hood.

Q9 · What is an RDD?

Resilient Distributed Dataset — immutable, partitioned, fault-tolerant collection.

- Resilient — rebuilt from lineage if lost
- Distributed — split across executors
- Immutable — every op makes a new RDD

When to still use RDDs: custom partitioners, unstructured text/binary, fine-grained control over physical execution.

Q10 · What is lineage?

The recorded chain of transformations that built a DataFrame. If a partition is lost, Spark recomputes only that partition from lineage instead of restarting the job — that's fault tolerance without replication.

```
df.explain(True)   # see all 4 plans
df.rdd.toDebugString()  # lineage
```

Q11 · What is lazy evaluation & why?

Transformations only build the plan; nothing runs until an action is called. This lets Catalyst see the ***whole pipeline at once*** and rewrite it — pushing filters to the scan, dropping unused columns, merging maps into one pass.

```
df = spark.read.parquet("/sales")
df = df.filter("amt > 100")   # lazy
df = df.select("id", "amt")  # lazy
df.count()      # ← job starts HERE
```

Everything above the action is just a plan. Catalyst then collapses filter + select into one scan that reads two columns and pushes the predicate to Parquet.

💡 Tip

Prefer DataFrames in PySpark. RDD lambdas run in a Python worker process, so every row is pickled across the JVM boundary — DataFrame code stays inside the JVM.

⭐ Interview Tip

Follow-up they love: ***"Is DataFrame faster than RDD?"*** — Yes, because of Catalyst optimisation and Tungsten's compact binary format, not because of the API itself.

Transformations vs Actions

Q12 · Difference?

A ***transformation*** returns a new DataFrame and is lazy. An ***action*** triggers execution and returns a value to the driver or writes data out.

Transformations (lazy)	Actions (eager)
select, filter, withColumn	show, count, collect
groupBy, agg, join	take, first, head
orderBy, distinct, union	write, save, saveAsTable
repartition, coalesce	foreach, toPandas

Q13 · Narrow vs Wide

Narrow — each output partition depends on ***one*** input partition. No data movement. Pipelined inside a stage.

Wide — output partitions pull from ***many*** input partitions → shuffle → new stage, disk + network cost.

Narrow ✓	Wide ✗ (shuffle)
select, filter, withColumn	groupByKey, reduceByKey
map, flatMap, union	join, distinct, repartition
coalesce (merging only)	orderBy, sortBy, window fns

SHUFFLE BOUNDARY = NEW STAGE

read → filter → withColumn ‖ shuffle ‖ groupBy → agg → write

Stage 1 (narrow ops pipelined together) | ***Stage 2*** (after the exchange)

Q14 · reduceByKey vs groupByKey

reduceByKey aggregates ***on the map side first***, then shuffles small partial results. groupByKey shuffles every raw record, then aggregates — far more network traffic and a common OOM cause.

```
rdd.reduceByKey(lambda a,b: a+b)
```

With DataFrames you don't choose — groupBy().agg() already does partial aggregation on the map side.

Q15 · collect() vs show() vs take()

- collect() — pulls ***all*** rows to the driver. Use only on tiny results, else driver OOM.
- show(n) — prints n rows (default 20), safe.
- take(n) — returns n rows as a list, reads only the partitions it needs.

⭐ Interview Tip

If asked ***"how do you know a shuffle happened?"*** — say: the DAG shows an Exchange node in `df.explain()`, and the Spark UI splits the job into a new stage with shuffle read/write bytes.

Bonus · Ops that secretly trigger a job

- ***inferSchema=True*** — an extra full scan
- ***pivot("col")*** without the value list
- ***checkpoint()*** — writes immediately
- ***df.rdd*** conversions and ***toPandas()***

SparkSession & Reading Data

Q16 · SparkSession vs SparkContext

SparkContext	SparkSession
Entry point for **RDD** API (Spark 1.x)	Unified entry point since **2.0**
Needed HiveContext / SQLContext separately	Wraps all of them in one object
One per JVM	Many sessions, isolated catalogs, one context

```
from pyspark.sql import SparkSession
spark = (SparkSession.builder
   .appName("sales_etl")
   .config("spark.sql.shuffle.partitions",
           "200")
   .getOrCreate())
sc = spark.sparkContext  # reachable
```

Q17 · inferSchema vs StructType

inferSchema=True makes Spark read the file ***an extra time*** to guess types — slow on big data and it can guess wrong (IDs become long, dates become strings).

In production always pass an explicit schema — one pass, stable types, and it fails loudly when the source changes.

```
from pyspark.sql.types import *
schema = StructType([
 StructField("id", IntegerType(), False),
 StructField("amt", DoubleType(), True),
 StructField("ts", TimestampType())])
```

Q18 · How do you read files?

```
df = (spark.read.format("csv")
      .option("header", True)
      .option("mode", "PERMISSIVE")
      .schema(schema)
      .load("/mnt/raw/sales/*.csv"))

df.write.mode("overwrite") \
  .partitionBy("order_date") \
  .parquet("/mnt/curated/sales")
```

Q19 · Read modes for bad records

Mode	Behaviour
PERMISSIVE	Default. Nulls the bad fields, keeps the row
DROPMALFORMED	Silently drops the bad row
FAILFAST	Throws immediately

Add `columnNameOfCorruptRecord` to capture the raw bad line into its own column for a quarantine table.

Q20 · Write modes

Mode	What it does
overwrite	Replaces existing data at path
append	Adds new files alongside
ignore	No-op if path exists
error / errorifexists	Default — throws if path exists

> 💡 **Tip**
>
> ***printSchema()*** before every join or write. Half of all "job failed in prod" stories are a silently changed source column type.

> ⭐ **Interview Tip**
>
> Asked ***"how do you handle a corrupt file in a folder of 500?"*** — badRecordsPath (Databricks) or PERMISSIVE + corrupt-record column, quarantine the rows, alert, and keep the pipeline running.

DataFrame Operations

Q21 · select vs selectExpr vs withColumn

```
# select — pick / rename / transform
df.select("id",
   col("amt").alias("amount"))

# selectExpr — SQL strings
df.selectExpr("id",
   "amt * 1.18 as amt_gst")

# withColumn — add / replace ONE col
df.withColumn("amt_gst",
   col("amt") * 1.18)
```

Trap: chaining 50 ***withColumn*** calls builds a deep plan and slows Catalyst. Use one select with all expressions, or withColumns({...}) (Spark 3.3+).

Q22 · Conditional logic

```
from pyspark.sql.functions \
     import when, col
df.withColumn("tier",
   when(col("amt") >= 10000, "GOLD")
  .when(col("amt") >= 5000, "SILVER")
  .otherwise("BRONZE"))
```

No otherwise() → unmatched rows become null, not an error. Interviewers ask this exact thing.

Q23 · distinct vs dropDuplicates

distinct() dedups on all columns. ***dropDuplicates(["id"])*** dedups on a subset and keeps an arbitrary row — pair it with a window if you need "keep the latest".

Q24 · Most-used functions cheat table

Need	Function
Rename	withColumnRenamed("a","b")
Drop columns	df.drop("c1","c2")
Cast type	col("amt").cast("double")
Split / explode array	split(), explode(), posexplode()
Dates	to_date, date_format, datediff, add_months
Strings	trim, upper, concat_ws, regexp_replace, substring
Stack rows	unionByName(df2, allowMissingColumns=True)

Q25 · union vs unionByName

union() matches by ***position*** — if column order differs you silently merge the wrong data. unionByName() matches by name and can fill missing columns with null. Always prefer it.

> ⭐ **Interview Tip**
>
> ***"Is a DataFrame mutable?"*** — No. Every operation returns a new DataFrame; `df.withColumn(...)` without reassigning changes nothing.

> 💡 **Tip**
>
> Learn ***explode*** and ***to_json / from_json*** properly. Nested JSON flattening is the single most common live-coding task in data engineering interviews.

Joins — The Make-or-Break Round

Q26 · Join types in PySpark

Type	Returns
inner	Only matching rows (default)
left / right	All rows from one side + matches
full / outer	Everything from both sides
left_semi	Left rows that have a match — no right columns
left_anti	Left rows with no match — great for "new records"
cross	Cartesian product — usually a bug

```
df1.join(df2, "cust_id", "left_anti")
# same key name → no duplicate column
df1.join(df2, df1.id == df2.cid, "inner")
```

Q27 · Join strategies Spark picks

Strategy	When
Broadcast Hash Join	One side < autoBroadcastJoinThreshold (10 MB default)
Sort Merge Join	Default for two large tables — shuffle + sort both sides
Shuffle Hash Join	One side much smaller but too big to broadcast
Broadcast Nested Loop	Non-equi joins — very slow

semi vs inner: semi never duplicates left rows when the right side has many matches. Use it for existence checks.

Q28 · What is a broadcast join?

The small table is shipped to **every executor**, so the big table never shuffles. Turns an expensive wide op into a narrow one.

```
from pyspark.sql.functions \
    import broadcast
big.join(broadcast(dim),
         "prod_id", "left")

spark.conf.set(
 "spark.sql.autoBroadcast"
 "JoinThreshold", 100*1024*1024)
```

Danger: broadcasting something too big → driver OOM or executor memory blowup. Set **-1** to disable it entirely.

Q29 · Duplicate rows after a join?

Almost always a non-unique key on the right side. Check with:

```
dim.groupBy("prod_id").count() \
   .filter("count > 1").show()
```

Fix by deduping the dimension first (window + row_number), or by using left_semi if you only needed a filter.

Q30 · Ambiguous column error?

Two DataFrames share a column name. Fix: join on the **string key** instead of a condition, or alias both sides — `a.alias("l")` then `col("l.id")`.

⭐ Interview Tip

The killer follow-up is **"your join takes 3 hours, what do you check?"** — order: (1) is one side broadcastable, (2) is the key skewed, (3) filter before joining, (4) are both sides already partitioned/bucketed on the key, (5) is AQE on.

Aggregations & Window Functions

Q31 · groupBy + agg

```
from pyspark.sql import functions as F
(df.groupBy("region", "month")
   .agg(F.sum("amt").alias("rev"),
        F.countDistinct("cust_id"),
        F.avg("amt").alias("aov"))
   .orderBy(F.desc("rev")))
```

count("col") skips nulls, ***count("*")*** counts rows. Classic gotcha question.

Q32 · pivot

```
df.groupBy("region").pivot("month",
   ["Jan","Feb","Mar"]).sum("amt")
```

Always pass the value list — without it Spark runs an extra job to find distinct values.

Q33 · What is a window function?

It computes a value ***per row*** using a group of related rows, without collapsing them the way groupBy does.

```
from pyspark.sql.window import Window
w = (Window.partitionBy("cust_id")
       .orderBy(F.desc("order_ts")))
# latest row per customer
df.withColumn("rn",
     F.row_number().over(w)) \
  .filter("rn = 1")
```

This "latest record per key" pattern is the most asked PySpark coding question. Memorise it.

Q34 · row_number vs rank vs dense_rank

Function	Ties	Scores 100,100,90
row_number()	Broken arbitrarily	1, 2, 3
rank()	Same rank, gap after	1, 1, 3
dense_rank()	Same rank, no gap	1, 1, 2

Other window fns: lag / lead (previous & next row), ntile(4) for quartiles, first / last, running sum with `rowsBetween`.

Q35 · Running total

```
w = (Window.partitionBy("cust")
  .orderBy("dt")
  .rowsBetween(
     Window.unboundedPreceding,
     Window.currentRow))
df.withColumn("run_tot",
              F.sum("amt").over(w))
```

rowsBetween counts physical rows; rangeBetween works on the value of the ordering column (used for time ranges).

💡 Tip

A window without partitionBy sends ***all rows to one partition*** — the job crawls and can OOM. Spark even logs "No Partition Defined for Window operation".

⭐ Interview Tip

Deduplication answer: ***row_number() over (partition by key order by ts desc) = 1***. Say it in SQL and PySpark — many interviewers ask for both.

Partitions & Shuffle

Q36 · What is a partition?

A logical chunk of the data on one executor. **Partitions = parallelism** — Spark can run at most one task per partition at a time.

```
df.rdd.getNumPartitions()
spark.conf.set(
  "spark.sql.shuffle.partitions", 200)
```

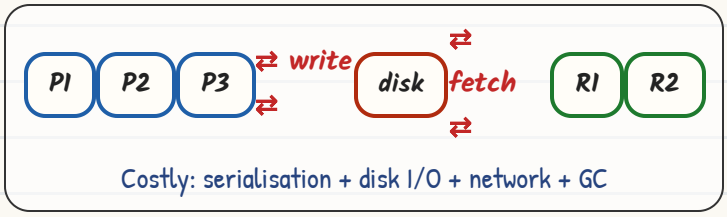
Rule of thumb: aim for partitions of **128 MB–200 MB**, and total partitions ≈ 2–4 × total executor cores.

Q37 · repartition vs coalesce

Point	repartition(n)	coalesce(n)
Shuffle	Yes, full	No
Direction	Up or down	Down only
Result	Even partitions	Can be skewed
Use for	Fixing skew, increasing parallelism	Cutting small output files

Q38 · What exactly is a shuffle?

Redistribution of data across executors so rows with the same key land together. Each map task writes **shuffle files to local disk**; reducers fetch them over the network.



Costly: serialisation + disk I/O + network + GC

Reduce shuffle by: filtering early, broadcasting small tables, pre-aggregating, bucketing on the join key, tuning shuffle partitions.

Q39 · partitionBy vs repartition

repartition() changes partitions in memory during processing. **write.partitionBy()** creates folders on disk (year=2026/month=08) so later reads can skip whole directories — partition pruning.

```
(df.repartition("order_date")
   .write.partitionBy("order_date")
   .parquet(path))
```

Doing both together avoids each partition folder getting one small file per task.

Q40 · What is AQE?

Adaptive Query Execution (on by default in Spark 3.2+). Uses runtime statistics to: coalesce shuffle partitions, switch sort-merge to broadcast, and split skewed partitions.

```
spark.sql.adaptive.enabled = true
spark.sql.adaptive.skewJoin.enabled = true
```

⭐ Interview Tip

"Default 200 shuffle partitions" is the number they want to hear. Then add: on a 10 GB shuffle that's 50 MB each which is fine, but on 2 TB it's 10 GB per task — spill and OOM. Tune it or let AQE coalesce.

Caching, Persist & Checkpoint

Q41 · cache() vs persist()

Same mechanism. **cache()** is shorthand for **persist(MEMORY_AND_DISK)** — the default storage level for DataFrames. persist() lets you choose the level.

Both are lazy — nothing is stored until the next action runs.

```
from pyspark import StorageLevel
df.cache()
df.persist(StorageLevel.MEMORY_ONLY)
df.count()          # materialises
df.unpersist()      # free it!
```

Q42 · Storage levels

Level	Behaviour
MEMORY_ONLY	RAM only; missing partitions recomputed
MEMORY_AND_DISK	Spills to disk instead of recomputing
MEMORY_ONLY_SER	Serialised — less RAM, more CPU
DISK_ONLY	Disk only
…_2	Replicated on 2 nodes

In PySpark, RDD data is already serialised, so **_SER** levels behave the same as their plain counterparts.

Q43 · When SHOULD you cache?

- ✓ The same DataFrame feeds **2+ actions** or branches
- ✓ Iterative ML / graph loops over one dataset
- ✓ An expensive join or aggregation reused downstream
- ✓ Interactive notebook exploration on a fixed slice

When NOT to

- ✗ Used once — caching only adds overhead
- ✗ Dataset far bigger than cluster memory
- ✗ Straight read → transform → write pipelines
- ✗ Cached and never unpersisted (evicts everyone else)

Q44 · cache vs checkpoint

Point	cache	checkpoint
Stored in	Executor memory / local disk	Reliable store (HDFS/S3)
Lineage	Kept	Truncated
Survives app	No	Yes

Checkpoint when lineage gets very long (loops, streaming) — otherwise the DAG itself becomes the bottleneck and recovery gets expensive.

```
sc.setCheckpointDir("/mnt/chk")
df.checkpoint()
```

> 💡 **Tip**
>
> Check the **Storage tab** in the Spark UI. If "Fraction Cached" is under 100%, your cache is being evicted and partitions are silently recomputed.

> ⭐ **Interview Tip**
>
> Say **"cache is a hint, not a guarantee"**. Under memory pressure Spark evicts cached blocks using LRU — with MEMORY_ONLY those partitions get recomputed from lineage.

UDFs, Spark SQL & Shared Variables

Q45 · Why are Python UDFs slow?

Rows leave the JVM, get ***pickled*** into a Python worker, are processed row by row, then serialised back. Catalyst treats the UDF as a black box — no predicate pushdown, no codegen through it.

```
# ✗ avoid when a built-in exists
@udf(StringType())
def upper_name(s): return s.upper()

# ✓ same result, 10x faster
df.withColumn("n", F.upper("name"))
```

Order of preference: built-in function → SQL expression → ***pandas UDF*** → plain Python UDF.

Q46 · What is a pandas UDF?

A ***vectorised*** UDF. Spark ships batches of rows to Python as Apache Arrow columnar data, your function works on a whole pandas Series at once — usually 3–100× faster than a row UDF.

```
from pyspark.sql.functions \
     import pandas_udf

@pandas_udf("double")
def gst(s: pd.Series) -> pd.Series:
    return s * 1.18

df.withColumn("amt_gst", gst("amt"))
```

Q47 · Spark SQL & temp views

```
df.createOrReplaceTempView("sales")
spark.sql("""SELECT region,
   SUM(amt) rev FROM sales
   GROUP BY region""")

df.createOrReplaceGlobalTempView(
   "g_sales")
# query as global_temp.g_sales
```

View type	Scope
Temp view	One SparkSession
Global temp view	All sessions in the app
Managed table	Metastore owns data + metadata
External table	Metastore owns metadata only — drop keeps files

Q48 · SQL or DataFrame API — which is faster?

Neither. Both compile to the same Catalyst logical plan and the same physical plan. Pick whichever is more readable for the team; mix them freely.

Q49 · Broadcast variable vs Accumulator

Broadcast	Accumulator
Read-only value cached on every executor	Write-only counter aggregated on the driver
Lookup maps, config, small dims	Counting bad records, metrics

```
bc = sc.broadcast({"IN":"India"})
acc = sc.accumulator(0)
```

Careful: accumulator values are only reliable inside ***actions*** — a retried or recomputed task can double count in transformations.

⭐ Interview Tip

When you say "I avoid UDFs", expect ***"but what if the logic can't be expressed with built-ins?"*** — answer: pandas UDF first; if it still must be a row UDF, keep it tiny, return a precise type, and filter the data down before applying it.

Nulls, Duplicates & Data Quality

Q50 · Handling nulls

```
df.na.drop("any")          # row w/ any null
df.na.drop("all")          # row all null
df.na.drop(subset=["id","amt"])
df.na.fill({"amt": 0, "city": "NA"})
df.na.replace(["N/A","-"], None)
```

thresh=2 keeps rows with at least 2 non-null values.

Q51 · Counting nulls per column

```
df.select([F.count(F.when(
   F.col(c).isNull(), c)).alias(c)
   for c in df.columns]).show()
```

Q52 · Null behaviour traps

`null = null` is ***not*** true — use `isNull()` or `eqNullSafe()`

- Any arithmetic with null returns null
- ***count("col")*** ignores nulls; ***count("*")*** doesn't
- sum / avg skip nulls silently — avg is over non-null rows only
- Nulls in a ***join key*** never match, they get dropped in an inner join
- orderBy puts nulls first ascending — use `asc_nulls_last()`

```
# null == null matches
a.join(b, a.k.eqNullSafe(b.k))
F.coalesce("discount", F.lit(0))
```

Q53 · Remove duplicates but keep the newest row

```
w = (Window.partitionBy("cust_id")
       .orderBy(F.desc("updated_at")))
clean = (df
   .withColumn("rn", F.row_number().over(w))
   .filter("rn = 1").drop("rn"))
```

Plain ***dropDuplicates(["cust_id"])*** keeps a random row — never use it when recency matters.

Q54 · Data quality checks you'd add

- ✓ Row count vs source (reconciliation)
- ✓ Primary key uniqueness + not null
- ✓ Accepted values / range checks on amounts
- ✓ Referential check with left_anti against the dim
- ✓ Freshness: max(event_ts) vs SLA

> 💡 **Tip**
>
> Quarantine, don't drop. Write failing rows to a ***_rejects*** table with a reason column — you'll be asked "how do you handle bad data?" and this is the senior answer.

> ⭐ **Interview Tip**
>
> Expect: ***"Your row count dropped 30% overnight — debug it."*** Check source counts, then joins (inner join on a null-heavy key), then filters on a column whose values changed casing or format.

File Formats & Storage

Q55 · Compare the formats

Format	Layout	Schema evolution	Best for
CSV / JSON	Row, text	None / weak	Raw landing zone, interchange
Avro	Row, binary	Excellent	Streaming, Kafka payloads, write-heavy
Parquet	**Columnar**	Add columns	Analytics — the default choice
ORC	Columnar	Add columns	Hive-heavy stacks, ACID in Hive
Delta	Parquet + JSON log	Full + enforcement	Lakehouse, ACID, updates, time travel

Q56 · Why is Parquet so fast?

- ✓ **Column pruning** — reads only the columns you select
- ✓ **Predicate pushdown** — row-group min/max stats skip whole blocks
- ✓ Per-column encoding (dictionary, RLE) + snappy compression
- ✓ Schema travels with the file, so no inference pass

Row group ≈ 128 MB block of rows; each holds column chunks with statistics. That's the unit Spark skips.

Q57 · Partitioning vs Bucketing

Point	Partitioning	Bucketing
On disk	Folders per value	Fixed number of files by hash
Helps	Filter pruning	Joins & aggregations — skips shuffle
Column	Low cardinality (date, region)	High cardinality (user_id)

```
df.write.bucketBy(50, "cust_id") \
  .sortBy("cust_id").saveAsTable("t")
```

Q58 · The small files problem

Thousands of tiny files kill performance: one task per file, huge metadata listing cost on S3/ADLS, and driver memory pressure. Caused by over-partitioning, high shuffle partitions on write, or frequent streaming micro-batches.

Fixes: coalesce/repartition before write · partition on a coarser column · **OPTIMIZE** / compaction job · `maxRecordsPerFile` · auto-optimize on Delta.

Q59 · Splittable compression

Codec	Splittable?
snappy	Yes inside Parquet/ORC — default
gzip	No on text — one huge file = one task
bzip2 / lz4 / zstd	Yes

⭐ **Interview Tip**

"Why not partition by customer_id?" — millions of tiny folders, catastrophic metadata overhead. Partition on something with tens-to-thousands of distinct values, and bucket or Z-order the high-cardinality key instead.

Delta Lake & Lakehouse Q&A

Q60 · What is Delta Lake?

A storage layer over Parquet that adds a ***transaction log*** (`_delta_log` of JSON commits + checkpoints). That log gives you ACID transactions, updates/deletes, schema enforcement and time travel on cheap object storage.

- ✓ ACID commits — no half-written tables
- ✓ UPSERT via MERGE (impossible in plain Parquet)
- ✓ Schema enforcement + controlled evolution
- ✓ Time travel & rollback by version or timestamp
- ✓ Unified batch + streaming source and sink

Q61 · MERGE / upsert

```
from delta.tables import DeltaTable
tgt = DeltaTable.forPath(spark, path)
(tgt.alias("t").merge(
   src.alias("s"), "t.id = s.id")
 .whenMatchedUpdateAll()
 .whenNotMatchedInsertAll()
 .execute())
```

This is how ***SCD Type 1*** is done. For ***SCD Type 2***: match on business key + `is_current`, close the old row with an end date, insert the new version.

Q62 · Time travel

```
spark.read.format("delta") \
  .option("versionAsOf", 12).load(path)

spark.sql("""SELECT * FROM sales
   TIMESTAMP AS OF '2026-08-01'""")
spark.sql("DESCRIBE HISTORY sales")
```

Retention is bounded by ***VACUUM*** (default 7 days). After vacuuming, old versions are gone.

Q63 · OPTIMIZE, Z-ORDER, VACUUM

Command	Purpose
OPTIMIZE	Compacts small files into ~1 GB files
ZORDER BY	Co-locates related values → better data skipping on filter columns
VACUUM	Deletes files no longer referenced by the log

```
OPTIMIZE sales ZORDER BY (cust_id);
VACUUM sales RETAIN 168 HOURS;
```

Q64 · Medallion architecture

Bronze keeps history so you can always replay. Silver is the conformed layer with quality rules applied. Gold is business-facing and usually pre-aggregated.

⭐ Interview Tip

"Delta vs Parquet in one line?" — Delta is Parquet plus a transaction log; the log is what buys you ACID, updates and time travel.

Structured Streaming

Q65 · What is Structured Streaming?

Streaming built on the DataFrame API. Spark treats the stream as an ***unbounded table*** that keeps growing; your query runs incrementally in micro-batches. Same code as batch — only read/write change.

```
stream = (spark.readStream
   .format("kafka")
   .option("subscribe", "orders")
   .load())

(stream.writeStream
   .format("delta")
   .outputMode("append")
   .option("checkpointLocation",
           "/chk/orders")
   .trigger(processingTime="1 min")
   .start())
```

Q66 · Output modes

Mode	Writes
append	Only new rows — no aggregation, or with a watermark
update	Only rows that changed this batch
complete	Entire result table each time — aggregations only

Q67 · Trigger types

- ***default*** — next batch as soon as the last finishes
- ***processingTime="5 minutes"*** — fixed interval
- ***availableNow*** — process all pending data, then stop (replaces once)
- ***continuous*** — experimental, ~1 ms latency

Q68 · Watermark — why do you need it?

Late events would force Spark to keep every window's state forever. A watermark says "I'll wait N minutes for late data, then close the window and drop its state".

```
(stream
  .withWatermark("event_time",
                 "10 minutes")
  .groupBy(
     F.window("event_time", "5 minutes"),
     "store_id")
  .agg(F.sum("amt")))
```

Event time = when it happened (in the data). ***Processing time*** = when Spark saw it. Always aggregate on event time.

Q69 · Checkpoint & exactly-once

The checkpoint directory stores ***offsets, state and commit logs***. On restart Spark resumes from the last committed offset. With a replayable source (Kafka) and an idempotent sink (Delta), you get exactly-once end to end.

- ✗ Never delete or share a checkpoint dir between queries
- ✗ Changing the query logic can break checkpoint compatibility

Q70 · Batch vs Streaming — key extras

Concern	Streaming adds
State	Kept across batches (RocksDB store)
Ordering	Late & out-of-order events
Throttling	maxOffsetsPerTrigger / maxFilesPerTrigger
Monitoring	Input rate vs processing rate, batch duration

⭐ Interview Tip

"Is Spark real streaming?" — Structured Streaming is micro-batch by default, so latency is seconds, not milliseconds. If they need true per-event latency, name Flink. Knowing the limit scores better than defending Spark.

Performance Tuning & Data Skew

Q71 · How do you optimise a slow Spark job?

1. **Filter and select early** — move less data through every stage.
2. **Kill unnecessary shuffles** — broadcast small dims, pre-aggregate, avoid repeated repartition.
3. **Right-size partitions** — 128–200 MB each; tune shuffle partitions or let AQE coalesce.
4. **Fix skew** — salting, AQE skew join, or isolate the hot keys.
5. **Use columnar formats** + partition pruning; drop CSV in the curated layer.
6. **Cache only reused** DataFrames, unpersist after.
7. **Replace UDFs** with built-ins or pandas UDFs.

Q72 · What is data skew?

A few keys hold most of the rows, so one or two tasks run for hours while the rest finish in seconds. In the Spark UI: max task duration ≫ median, huge shuffle read on one task.

Q73 · How do you fix it?

- ✓ **Salting** — add a random suffix to the hot key, join, then aggregate away the salt
- ✓ Broadcast the smaller side to skip the shuffle entirely
- ✓ Enable **AQE skew join** — Spark splits the oversized partitions itself
- ✓ Filter out junk keys (null / "UNKNOWN" / default IDs) and handle them separately

```
big = big.withColumn("k_salt",
   F.concat_ws("_", "k",
     (F.rand()*10).cast("int")))
# explode small side 10x on same salt
```

Q74 · Memory config you should know

Setting	Meaning
spark.executor.memory	JVM heap per executor
spark.executor.cores	Task slots per executor (4–5 is the sweet spot)
spark.executor.memoryOverhead	Off-heap: Python workers, shuffle buffers
spark.memory.fraction	Share used for execution + storage (0.6)
spark.sql.shuffle.partitions	Partitions after a shuffle (200)

💡 **Tip**

PySpark OOM is often overhead, not heap. Python UDF workers live outside the JVM heap — raise memoryOverhead before raising executor memory.

⭐ **Interview Tip**

Don't list configs first. Say: **"I'd open the Spark UI, find the slowest stage, and check whether it's skew, spill, or too many small files."** Diagnosis before tuning.

Debugging & The Spark UI

Q75 · How do you debug a failing job?

1. Read the **real** error — scroll past the Py4J wrapper to the JVM stack trace and the failed stage ID.
2. Open the **Spark UI** → Stages → find the failed / slowest stage.
3. Check the task **summary metrics**: min / median / max duration, shuffle read, spill.
4. Check **executor logs** for OOM / GC / killed containers.
5. Run **df.explain("formatted")** to confirm the plan matches what you expected.
6. Reproduce on a **sample** (limit / sampleBy) before rerunning on TBs.

Q76 · Spark UI tabs

Tab	What you look for
Jobs	How many actions ran; failed jobs
Stages	Skew (max ≫ median), shuffle read/write, spill
Storage	Cached DataFrames, fraction cached
SQL	Query plan, rows scanned, join strategy chosen
Executors	GC time, dead executors, task distribution

Spill (memory / disk) in the Stages tab is your single best signal that partitions are too big.

Q77 · Errors you will be asked about

Error	Real cause	Fix
OutOfMemoryError: Java heap space (driver)	collect() / toPandas() on a big DataFrame, or a huge broadcast	Aggregate first, write to storage, cap broadcast threshold
Container killed by YARN, exceeding memory limits	Off-heap overhead — Python workers, shuffle buffers	Raise memoryOverhead, fewer cores per executor
FetchFailedException	Executor died mid-shuffle, or shuffle block too large	Fix skew, more partitions, enable external shuffle service
AnalysisException: cannot resolve 'col'	Typo, wrong case, or column dropped upstream	printSchema(), check the plan before the failure
Task not serializable	Closure captures a non-serialisable object (a connection, the session)	Create it inside the executor with mapPartitions
Job runs forever, one task at 99%	Data skew on the join / group key	Salting, AQE skew join, broadcast

💡 **Tip**

mapPartitions over **map** when you need a DB connection or model object — one setup per partition instead of per row.

⭐ **Interview Tip**

Have **one real war story** ready: the symptom, what you saw in the UI, what you changed, and the before/after runtime. That single story beats twenty memorised definitions.

Rapid-Fire Round

Q78–Q89 · One-line answers

Q. map vs flatMap?
map gives 1 output per input; flatMap can give 0, 1 or many (splitting a line into words).

Q. cache vs persist?
cache = persist(MEMORY_AND_DISK); persist lets you pick the level.

Q. Why is Spark lazy?
So Catalyst can optimise the whole DAG before running anything.

Q. Default parallelism?
Number of cores in the cluster (standalone/YARN); shuffle output defaults to 200.

Q. What is a DAG?
Directed Acyclic Graph of transformations — no cycles, so it can always be recomputed.

Q. Dynamic allocation?
Spark adds/removes executors based on pending tasks. Needs an external shuffle service.

Q. Speculative execution?
Re-runs straggler tasks on another node and takes whichever finishes first.

Q. coalesce(n) vs the coalesce() function?
coalesce(n) reduces partitions; F.coalesce(a,b) returns the first non-null column value. Same word, unrelated jobs.

Q. External shuffle service?
A separate process that serves shuffle files, so executors can be removed without losing them — required for dynamic allocation.

Q. What is Py4J?
The bridge that lets Python drive the JVM SparkContext.

Q. limit() vs take()?
limit returns a DataFrame (lazy); take returns rows to the driver (action).

Q. sort vs orderBy?
Identical in DataFrames — orderBy is an alias. Both do a full shuffle-sort.

Q. sortWithinPartitions?
Sorts inside each partition only, no shuffle — cheap when global order isn't needed.

Q. What does explain() show?
Parsed → Analyzed → Optimized logical plan → Physical plan.

Q. Immutable — why does it matter?
Enables lineage-based recovery and safe parallelism without locks.

Q. toPandas() risk?
Pulls the whole DataFrame into driver memory. Use Arrow + only on small results.

Q. spark-submit essentials?
--master, --deploy-mode, --num-executors, --executor-memory, --executor-cores, --conf.

Scenario questions they love

- "You must join a 5 TB fact with a 20 MB dim — how?" → broadcast the dim
- "Daily file dropped twice — avoid duplicates?" → MERGE on the key, or dedupe with a window
- "Only 3 of 200 tasks are slow" → skew, salt or AQE
- "Output has 10,000 files" → coalesce before write + compaction
- "Job succeeded but the numbers are wrong" → join fan-out from a non-unique key

⭐ Interview Tip

In rapid-fire rounds, answer in ***one sentence and stop***. Rambling invites a harder follow-up; a crisp answer makes the panel move on.

Coding Round + 30-Day Plan

Q90 · Second highest salary per dept

```
w = (Window.partitionBy("dept")
       .orderBy(F.desc("salary")))
(df.withColumn("rk",
      F.dense_rank().over(w))
   .filter("rk = 2"))
```

Q91 · Word count

```
(df.select(F.explode(F.split("line"," "))
     .alias("w"))
   .groupBy("w").count()
   .orderBy(F.desc("count")))
```

Q92 · Flatten nested JSON

```
df.select("id", "addr.city",
   F.explode("items").alias("item")) \
  .select("id","city","item.sku","item.qty")
```

Q93 · Customers with no orders

```
cust.join(orders, "cust_id", "left_anti")
```

Q94 · Month-over-month growth

```
w = (Window.partitionBy("region")
       .orderBy("mth"))
(m.withColumn("prev", F.lag("rev").over(w))
  .withColumn("growth_pct",
     (F.col("rev") - F.col("prev"))
      / F.col("prev") * 100))
```

Q95 · Incremental load pattern

```
last = spark.sql("SELECT MAX(ts) FROM tgt")
new = src.filter(F.col("ts") > last)
# then MERGE new into tgt on the key
```

30-DAY PREP PLAN — one hour a day is enough

WEEK 1 · Core

- Architecture, DAG, jobs/stages/tasks
- RDD vs DataFrame
- Transformations vs actions
- Read/write + schemas

WEEK 2 · SQL muscle

- Joins + join strategies
- Window functions daily
- Nulls, dedup, pivots
- 20 coding problems

WEEK 3 · Performance

- Shuffle, partitions, AQE
- Skew + salting
- Cache vs checkpoint
- Read the Spark UI on a real job

WEEK 4 · Project

- Delta + medallion pipeline
- Streaming from Kafka
- Mock interviews out loud
- Polish your war stories

Answer framework for any Spark question

Define it → Why it exists → Code / config → Real example

💡 Key Takeaway

Almost every advanced question reduces to one thing: ***how much data moves, and when***. Answer through that lens and you'll sound senior.

You've got 95 questions covered 🎉

Now go build a pipeline and break it — that's where the real answers come from.